МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Информационных систем и технологий
Специальность 6-05-0611-01 «Информационные системы и технологии»

Дисциплина Операционные системы

Лабораторная работа № 3
УПРАВЛЕНИЕ ПРОЦЕССАМИ

Студент 3 курса 2 группы
Марушко Т. Ф.
Проверил: Ассистент Савельева М. Г.

Минск, 2026

Содержание

Введение....3
1 Выполнение заданий на ОС *Linux*....4
1.1 Подготовка среды....4
1.2 Уровень 4–5: базовые операции с процессами....4
1.3 Уровень 7–8: *IPC* через анонимный канал....8
1.4 Уровень 9–10: обработка сигналов *SIGINT*....10
1.5 Вывод по разделу....11
2 Выполнение заданий на ОС *Windows*....12
2.1 Подготовка среды и сборка....12
2.2 Уровень 4–5: создание процессов (*CreateProcess*)....12
2.3 Уровень 7–8: *IPC* через канал и *STARTUPINFO*....18
2.4 Уровень 9–10: передача через *STARTUPINFO*....20
2.5 Вывод по разделу....21
Заключение....22

Введение

Цель работы – получить навыки управления процессами в *Windows* и *Linux*: создание, контроль, обмен данными между процессами, передача через *STARTUPINFO*.

Задачи:

- написать *Lab*-03*x*: цикл 500 мс, вывод *PID*, аргумент или *ITER_NUM*;
- реализовать *Lab*-03*a*: три дочерних процесса *CreateProcess* разными способами;
- реализовать *Lab*-03*b*: глобальная и локальная *ITER_NUM*;
- написать *Lab*-03*c*: список процессов через *Tool Help Library* (*PID*, *PPID*, имя);
- реализовать *Lab*-03*d-client* / *Lab*-03*d-server*: поиск чисел, взаимно простых с *M*, через анонимный канал и *STARTUPINFO*;
- реализовать *Lab*-03*hello-parent* / *Lab*-03*hello-child*: передача строки через *STARTUPINFO* (*lpTitle* + *CREATE_NEW_CONSOLE*);
- написать аналоги для *Linux*: *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*d-client*, *Lab*-03*d-server*, *Lab*-03*sig* (устойчивость к *SIGINT*);
- проверить все программы, сделать скриншоты и выводы для уровней 4–5, 7–8, 9–10.

Работа выполнена на *Linux* (*Arch ARM*, *GCC* 16.1.1) и *Windows* (*Windows* 11 *ARM*, *MSVC* 19.42). Скриншоты выполнены в терминале. На рисунке 1.1 – сборка *Linux*, на рисунке 2.1 – сборка *Windows*.

1 Выполнение заданий на ОС *Linux*

1.1 Подготовка среды

Linux запущен в *VMware Fusion* (*Arch ARM*, *KDE Plasma*). Пользователь *User*-6*d*4*ec*851. Исходники в ~/*OS*3. Сборка показана на рисунке 1.1.

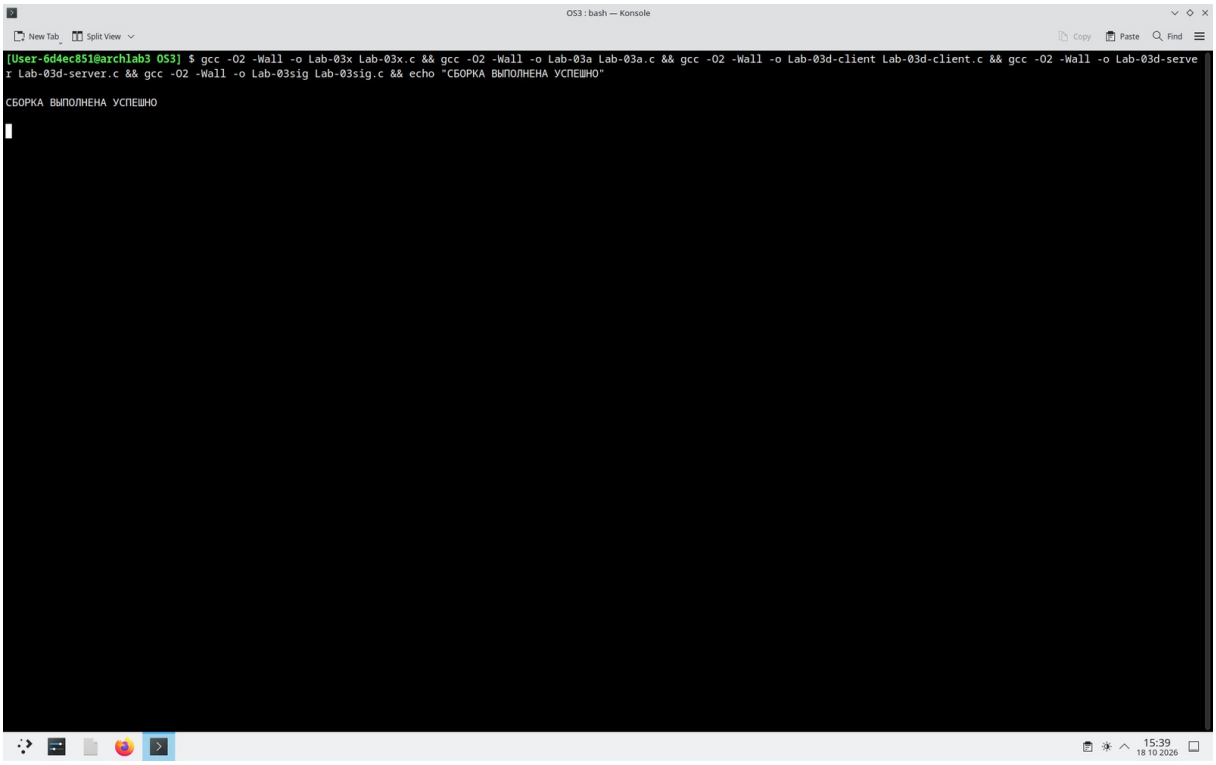


Рисунок 1.1 – Сборка *Linux*-приложений: *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*d-client*, *Lab*-03*d-server*, *Lab*-03*sig*

Все пять программ собрались без ошибок (*gcc -O*2 *-Wall*). Исправлены баги: в *Lab*-03*d-server* парсер ждал 3 токена вместо 4, в *Lab*-03*sig* имя программы было *Lab*-03*c* вместо *Lab*-03*sig*. Среда готова.

1.2 Уровень 4–5: базовые операции с процессами

Проверяется создание процессов, передача параметров, ожидание завершения, анализ через /*proc* и *ps*.

Lab-03*x* запущен с аргументом (5 итераций), через *ITER_NUM*=7, без параметров (ошибка). Результаты на рисунках 1.2, 1.3, 1.4.

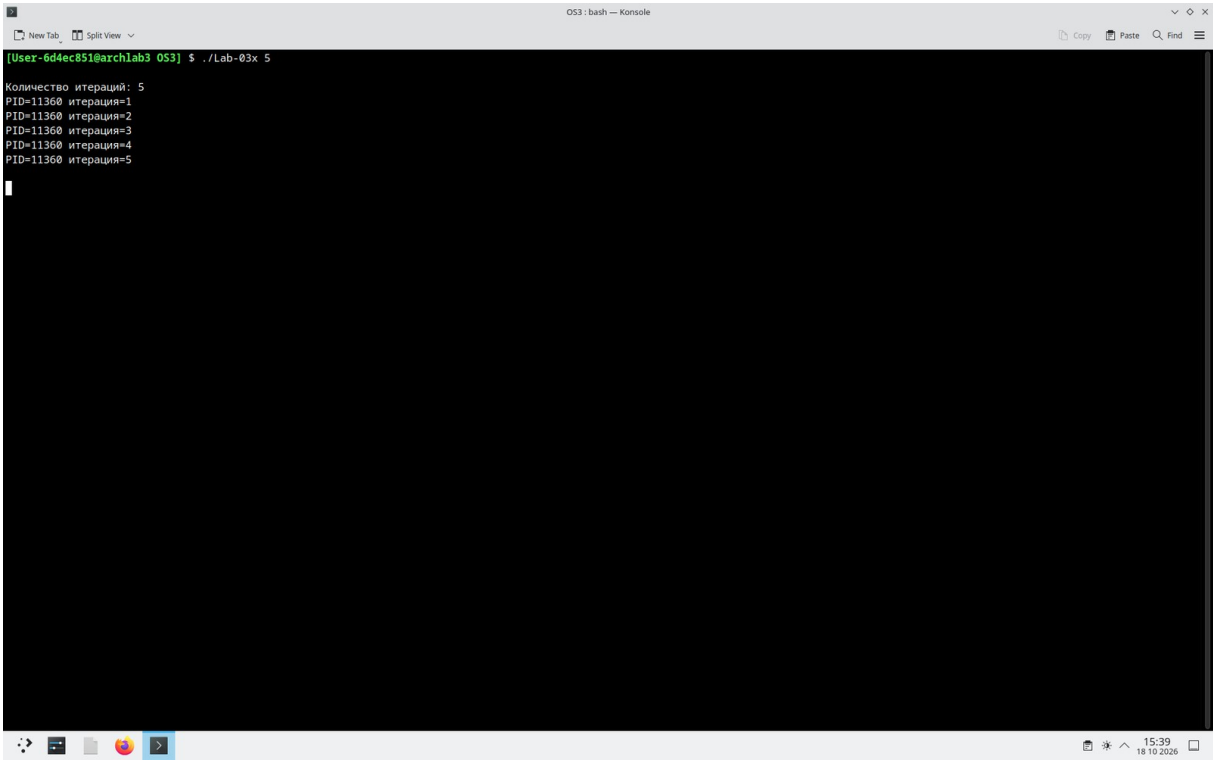


Рисунок 1.2 – *Lab*-03*x* с аргументом: 5 итераций, вывод *PID*

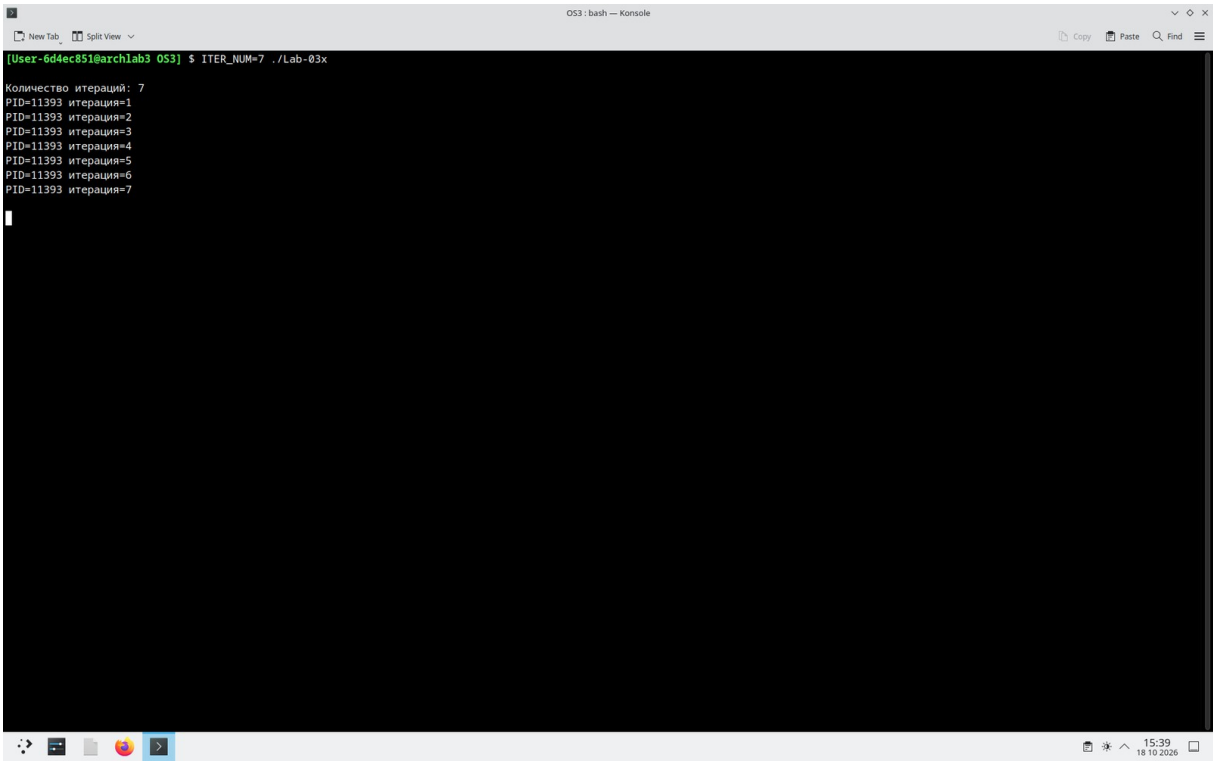


Рисунок 1.3 – *Lab*-03*x* через *ITER_NUM*=7: 7 итераций

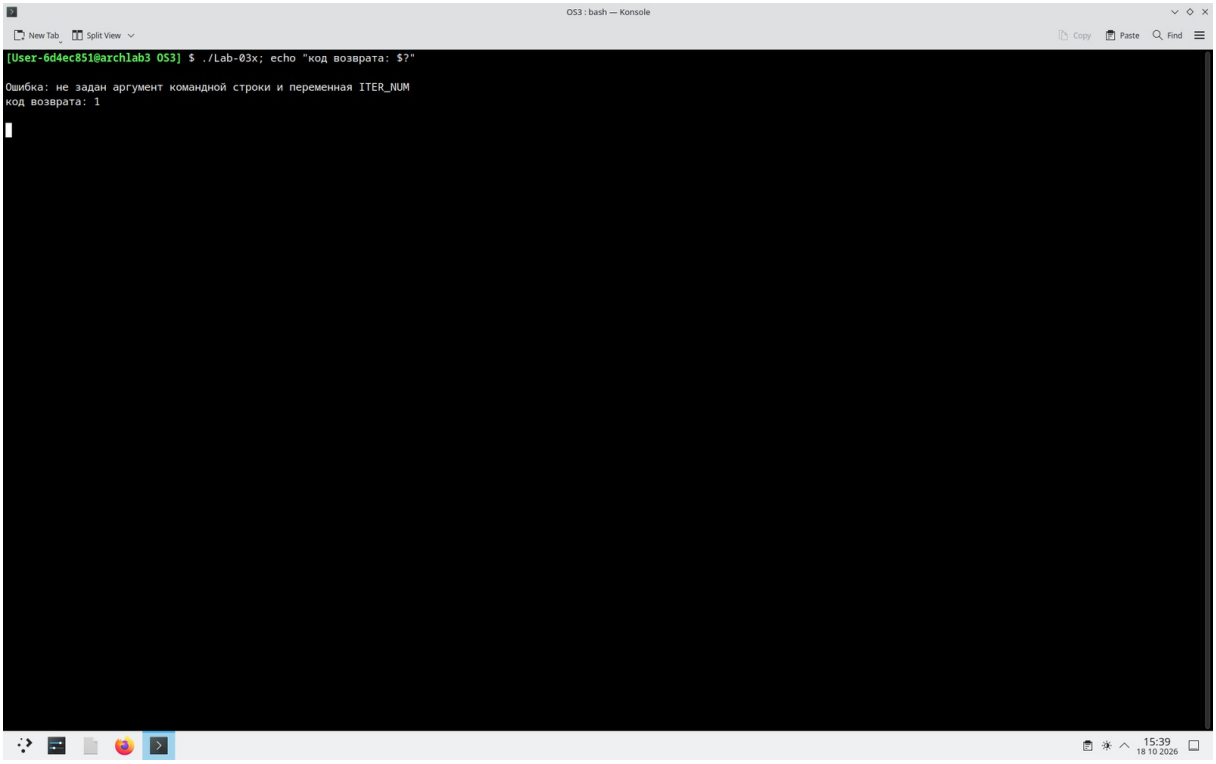


Рисунок 1.4 – *Lab*-03*x* без параметров: ошибка, код возврата 1

Lab-03*a* создаёт два дочерних *Lab*-03*x*: первый – аргумент 5, второй – *ITER_NUM*=8. Родитель ждёт оба (*waitpid*). Параллельная работа на рисунке 1.5.

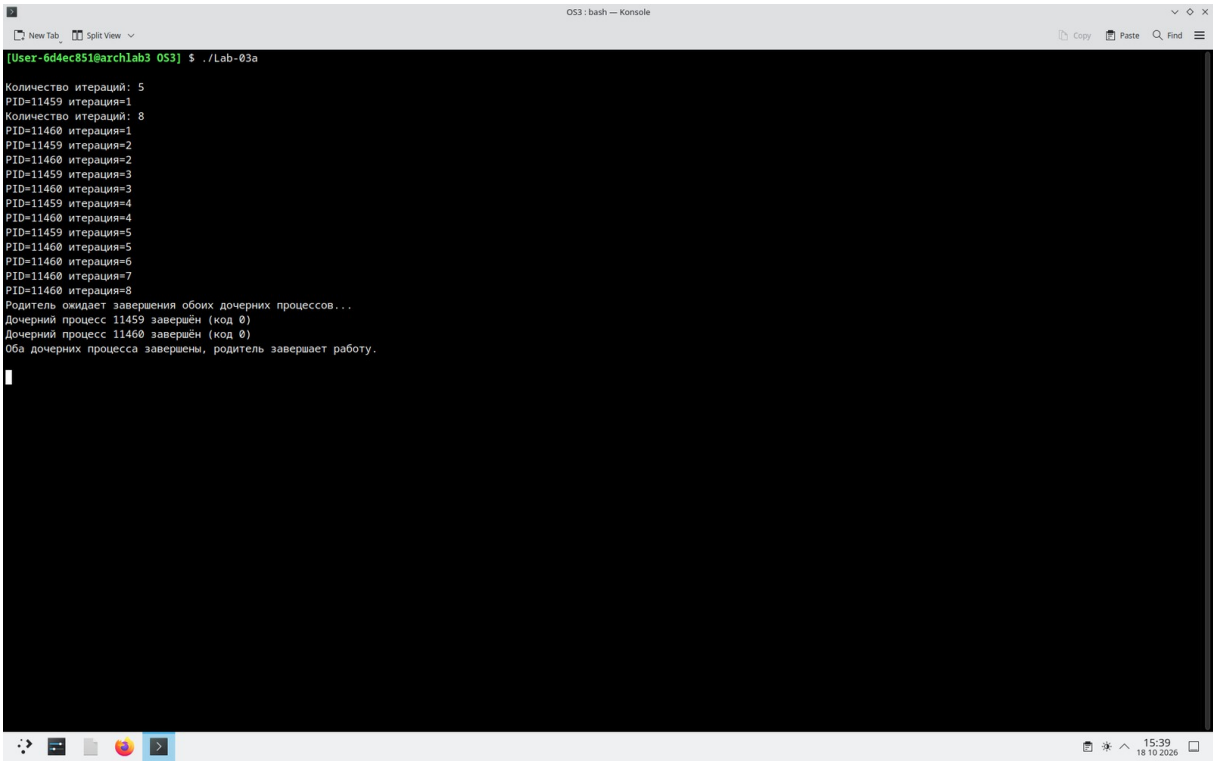


Рисунок 1.5 – *Lab*-03*a*: два дочерних процесса работают параллельно, родитель ждёт

Анализ дерева процессов: *ps -ef* и */proc*. Дерево на рисунке 1.6 (родитель *Lab*-03*a*, два дочерних *Lab*-03*x*, состояния *S*/*R*, *PPID*). */proc/<PID>/status* на

рисунке 1.7 (*State*, *Pid*, *PPid*, *Uid*, *Gid*). Содержимое */proc/<PID>* на рисунке 1.8 (*fd*, *environ*, *cmdline*, *status*). Проверки пройдены.

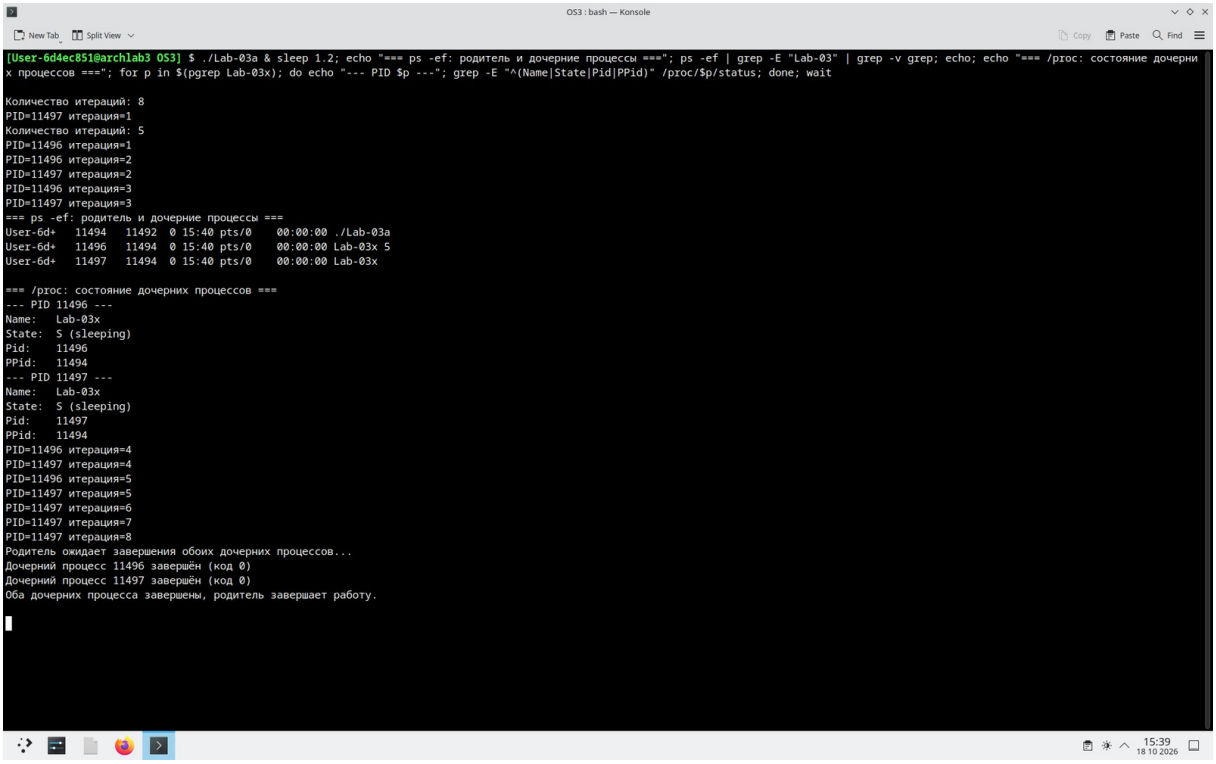


Рисунок 1.6 – Дерево процессов: *ps -ef* показывает родительские связи (*PPID*)

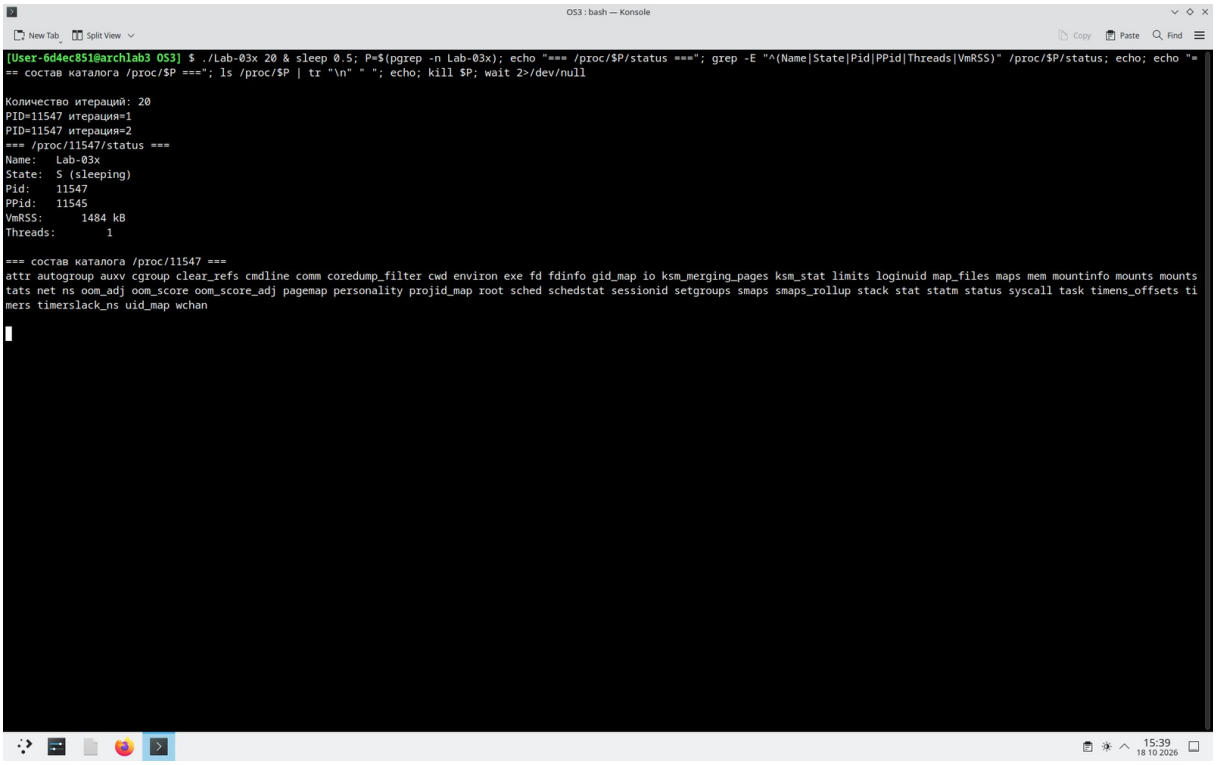


Рисунок 1.7 – Информация из */proc/<PID>/status*: *State*, *Pid*, *PPid*, *Uid*, *Gid*

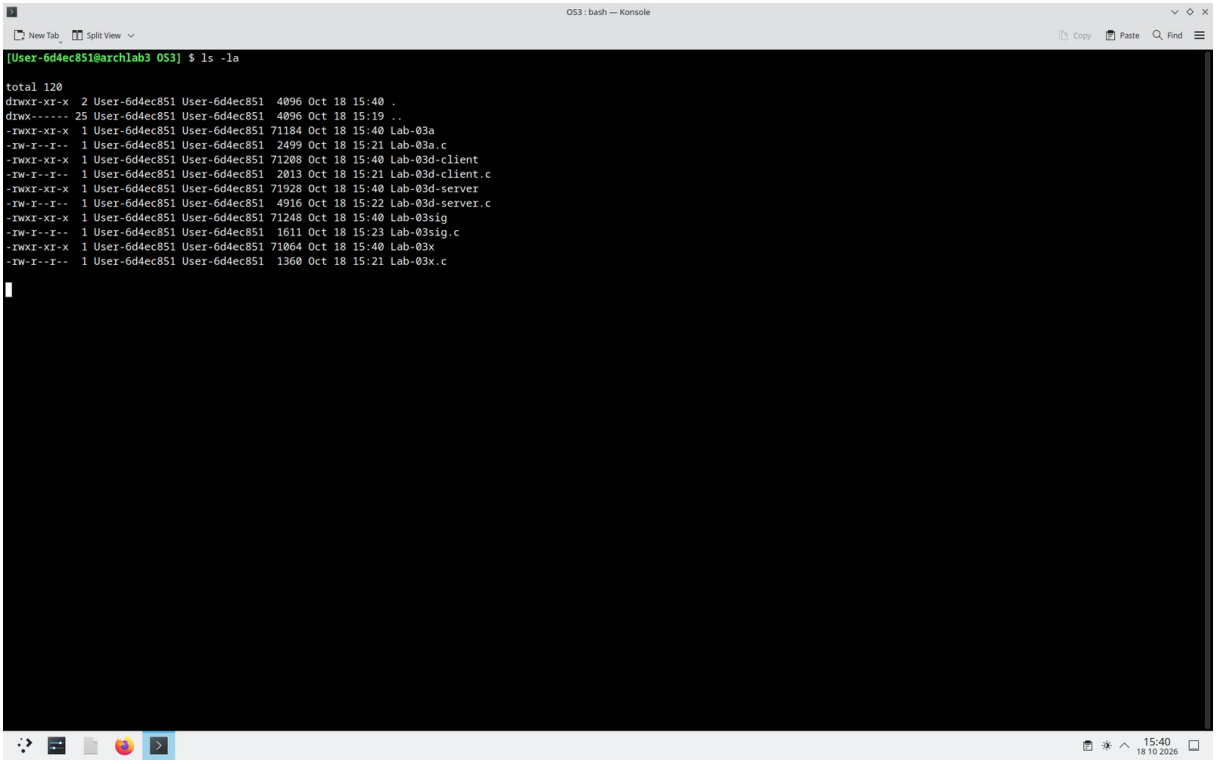


Рисунок 1.8 – Содержимое каталога /*proc*/<*PID*>: *fd*, *environ*, *cmdline*, *status*

Уровень 4–5 на *Linux* выполнен: все программы работают, параметры передаются, информация получена.

1.3 Уровень 7–8: *IPC* через анонимный канал

Lab-03*d-client* ищет числа, взаимно простые с *M* в [*L*, *R*], шлёт результат в *stdout* (перенаправлен в *pipe*). *Lab*-03*d-server* создаёт *pipe*, делит диапазон на *K* частей, запускает *K* клиентов, передаёт поддиапазоны аргументами, читает результаты, выводит список и время. Результаты на рисунках 1.9, 1.10, 1.11.

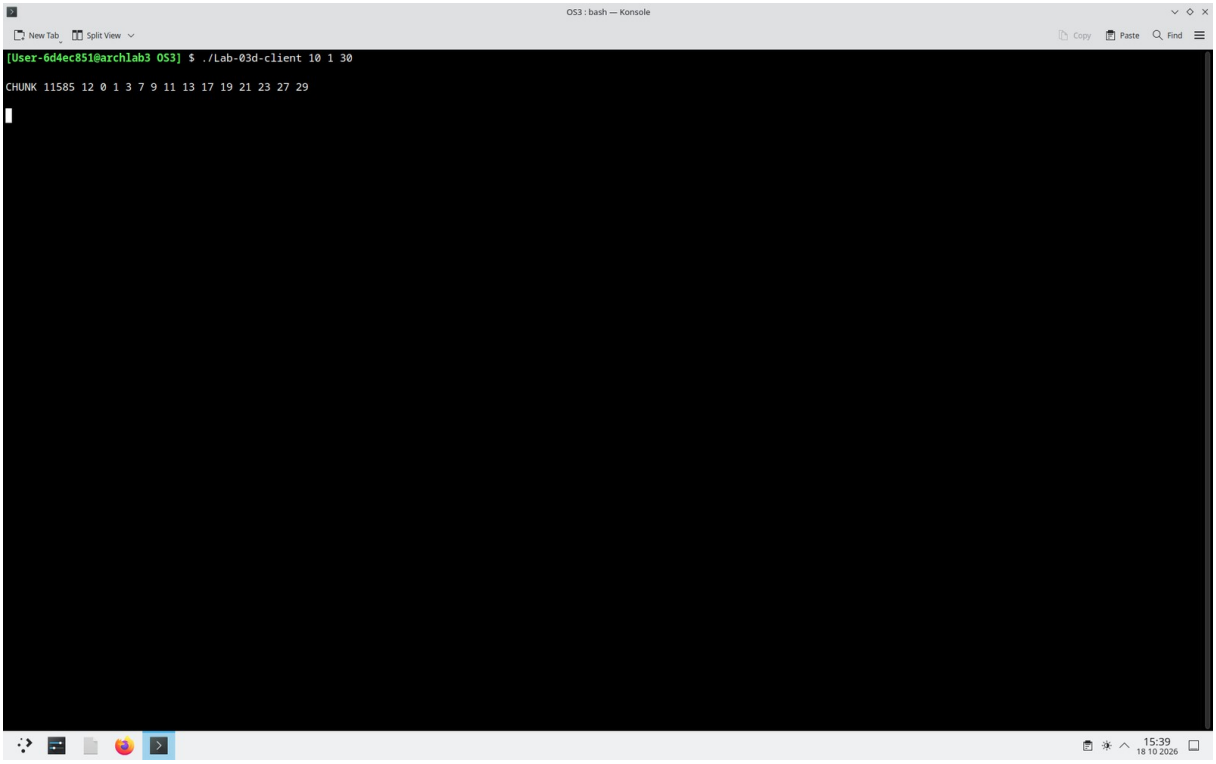


Рисунок 1.9 – *Lab*-03*d-client*: *M*=10, диапазон 1–30, найдены числа, вывод в канал

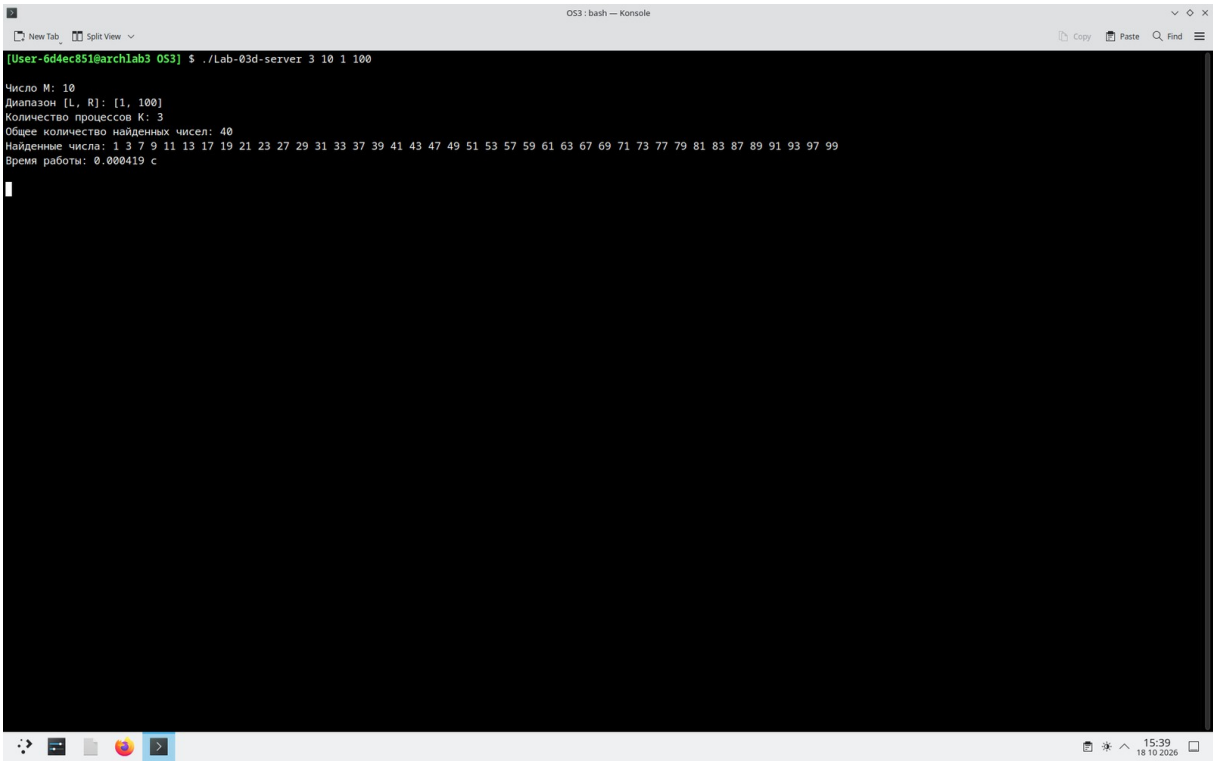


Рисунок 1.10 – *Lab*-03*d-server K*=3, *M*=10, 1–100: 40 чисел, время 0.004 с

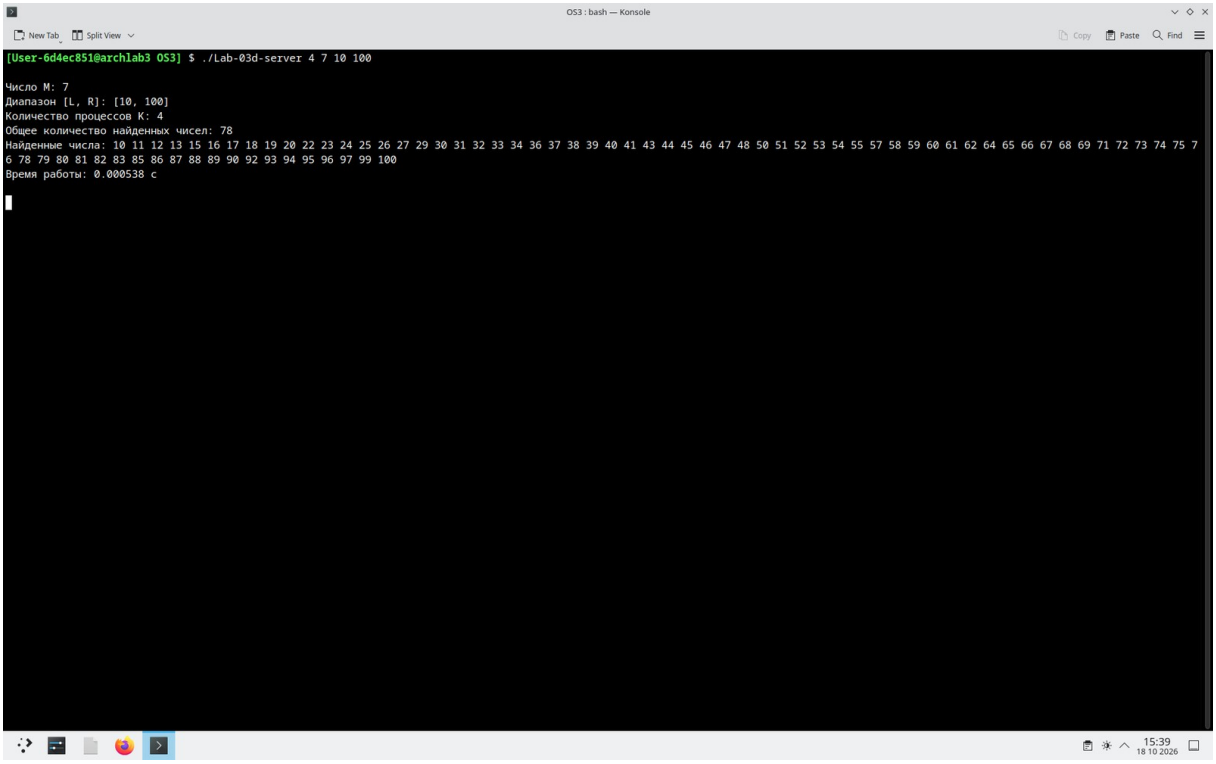


Рисунок 1.11 – *Lab*-03*d-server K*=4, *M*=7, 10–100: результат с 4 поддиапазонами

Сервер собирает результаты от всех детей, сортирует, выводит полную последовательность. Время через *clock_gettime*. Уровень 7–8 на *Linux* выполнен.

1.4 Уровень 9–10: обработка сигналов *SIGINT*

Lab-03*sig* ставит обработчик *SIGINT*, переустанавливает себя, выводит уведомление, не завершает программу. Цикл работает дальше, статус каждую секунду. Остановить можно только *SIGTERM* или *SIGKILL*. Демонстрация на рисунке 1.12.

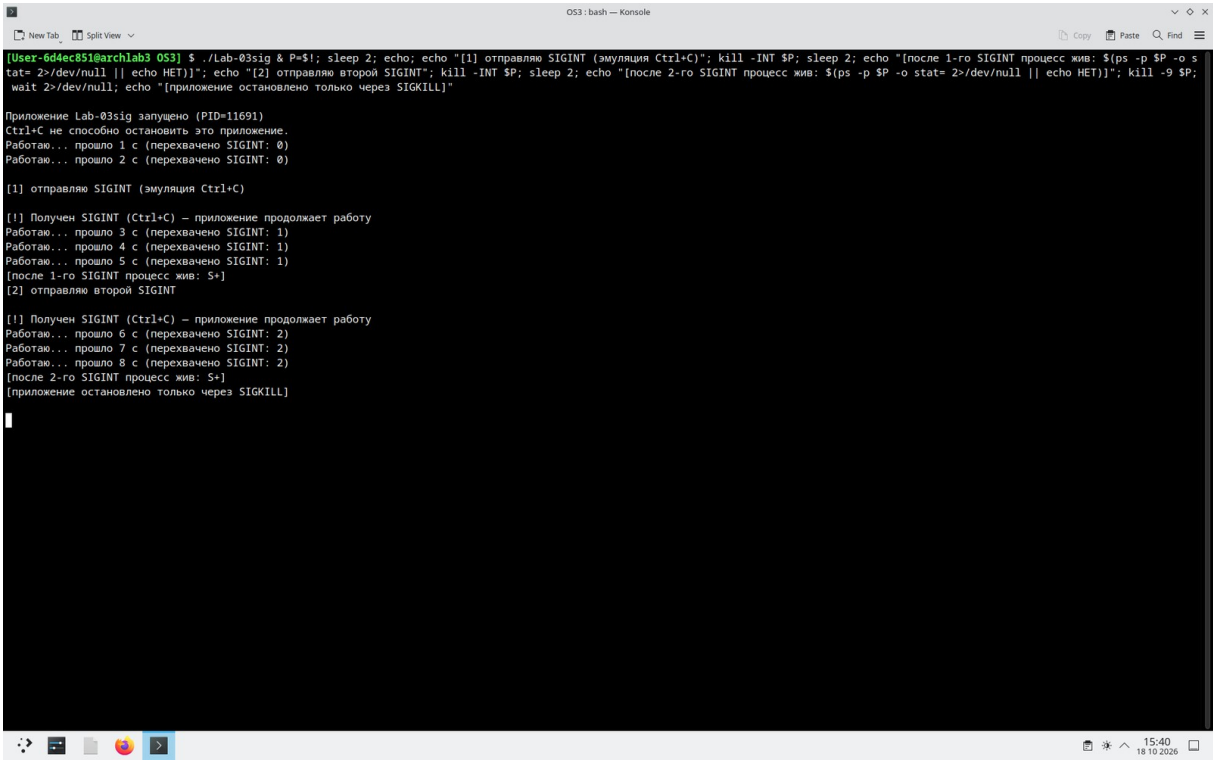


Рисунок 1.12 – *Lab*-03*sig*: два *Ctrl+C* (*SIGINT*) перехвачены, программа жива, завершается только *SIGKILL*

Программа перехватывает *SIGINT*, увеличивает счётчик, продолжает работу. Уровень 9–10 на *Linux* выполнен.

1.5 Вывод по разделу

На *Linux* реализованы: *Lab*-03*x* (цикл с аргументом/окружением), *Lab*-03*a* (*fork+exec* двух детей), *Lab*-03*d-client/server* (*pipe+fork*, деление диапазона, сбор результатов), *Lab*-03*sig* (перехват *SIGINT*, неубиваемость по *Ctrl+C*). Все собраны *GCC* 16.1.1, протестированы, скриншоты сделаны. Все уровни (4–5, 7–8, 9–10) выполнены.

2 Выполнение заданий на ОС *Windows*

2.1 Подготовка среды и сборка

Windows 11 *ARM* в *VMware Fusion*. Компилятор *MSVC* (*cl.exe*), *ARM*64 (*vcvarsall.bat arm*64). Кириллица в *cmd* (*cp*866) через *console_ru.h* (*UTF*-8 → *OEM* 866). Сборка на рисунке 2.1.



Рисунок 2.1 – Сборка 8 *Windows*-приложений: *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*b*, *Lab*-03*c*, *Lab*-03*d-client*, *Lab*-03*d-server*, *Lab*-03*hello-parent*, *Lab*-03*hello-child*

Все 8 программ собрались без ошибок. Среда готова.

2.2 Уровень 4–5: создание процессов (*CreateProcess*)

Lab-03*x*: цикл 500 мс, вывод *PID*. Итерации из аргумента; если нет – из *ITER_NUM*; если нет – *ExitProcess*(1). Результаты на рисунках 2.2, 2.3, 2.4.

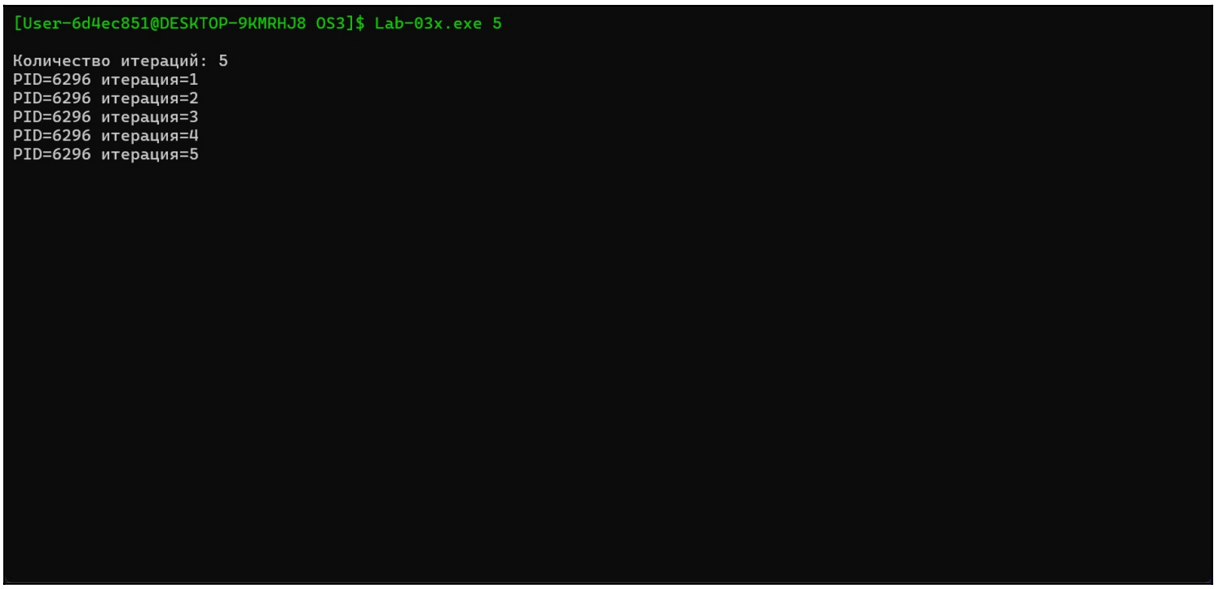


Рисунок 2.2 – *Lab*-03*x.exe* 5: 5 итераций, вывод *PID*

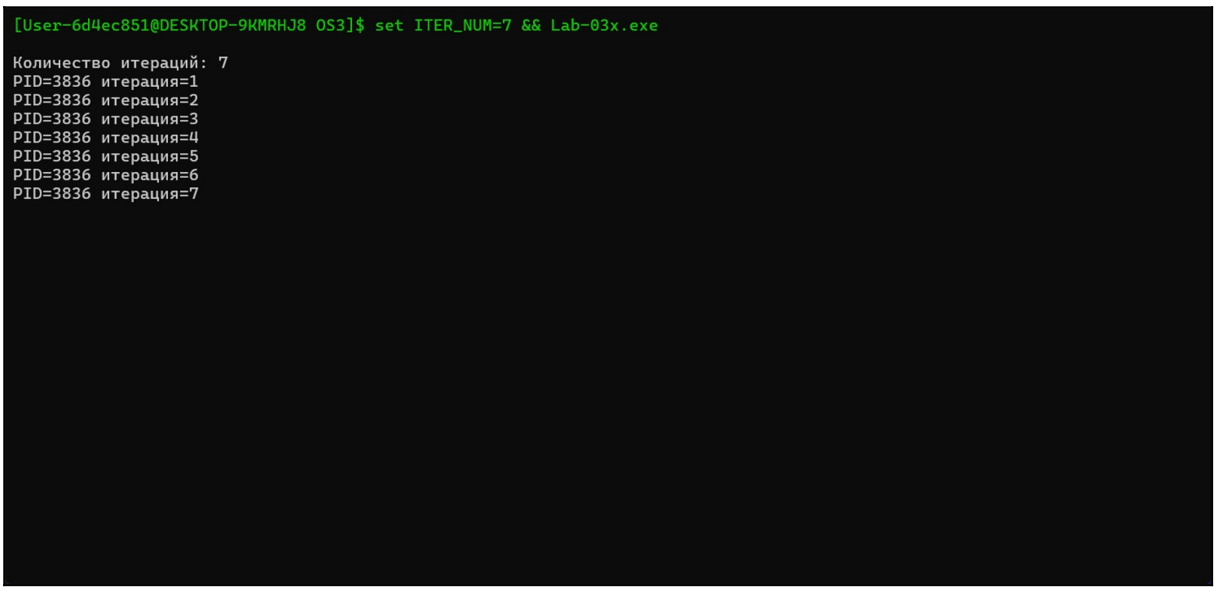


Рисунок 2.3 – *Lab*-03*x.exe* через *set ITER_NUM*=7: 7 итераций

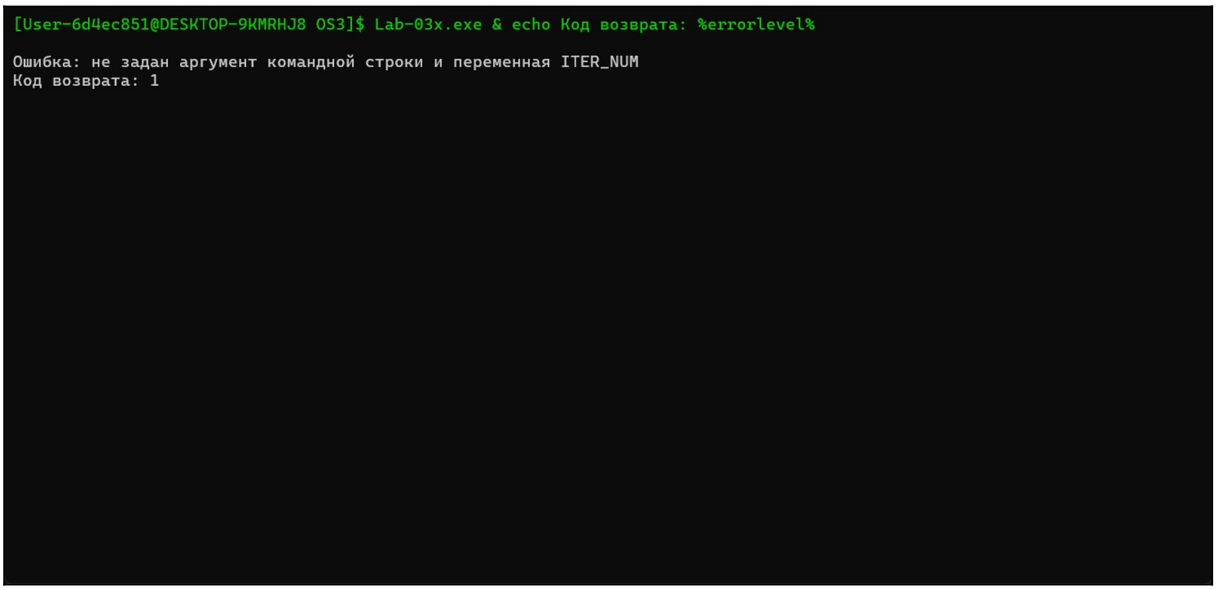


Рисунок 2.4 – *Lab*-03*x.exe* без параметров: ошибка, код возврата 1

Lab-03*a*: три *CreateProcess* для *Lab*-03*x.exe* разными способами: (1) только *lpApplicationName*, (2) только *lpCommandLine*, (3) *lpApplicationName* + *lpCommandLine* с пробелом. Проверено: (1) – ошибка 2 (файл не найден), (2) и (3) – работают. На рисунке 2.5.

Приложение *Lab*-03*a* аргументов командной строки не принимает: количество итераций дочерним процессам задано константой 5, что соответствует структуре вызова из методички. На рисунке 2.5 зафиксирован снимок, сделанный до унификации, когда аргумент ещё принимался.

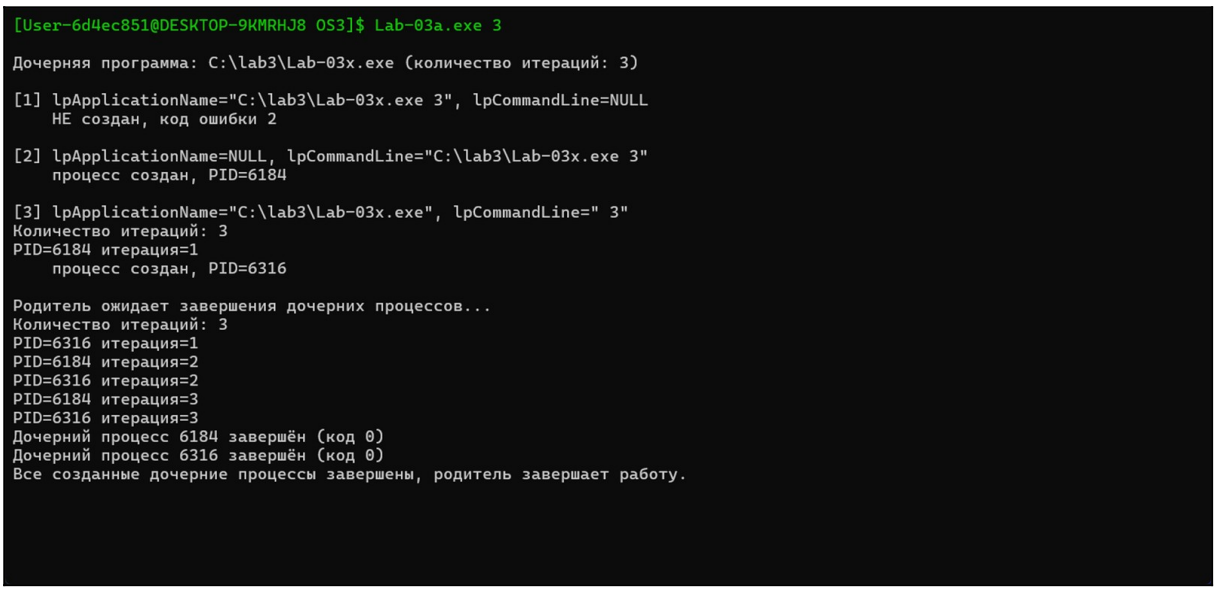


Рисунок 2.5 – *Lab*-03*a*.*exe* 3: два успешных процесса (2 и 3), первый – ошибка 2

Lab-03*b*: переменные окружения. (1) Только *exe* – глобальная *ITER_NUM* (нет → ошибка). (2) *lpCommandLine* с аргументом – работает. (3) *SetEnvironmentVariable*(«*ITER_NUM*», «3») перед вызовом – локальная переменная наследуется, итерации = 3. На рисунке 2.6.

Приложение *Lab*-03*b* также не принимает аргументов командной строки: итерации определяются переменной окружения *ITER_NUM*, локальное значение равно 3 и отличается от глобального. Снимок на рисунке 2.6 сделан до унификации структуры вызова.

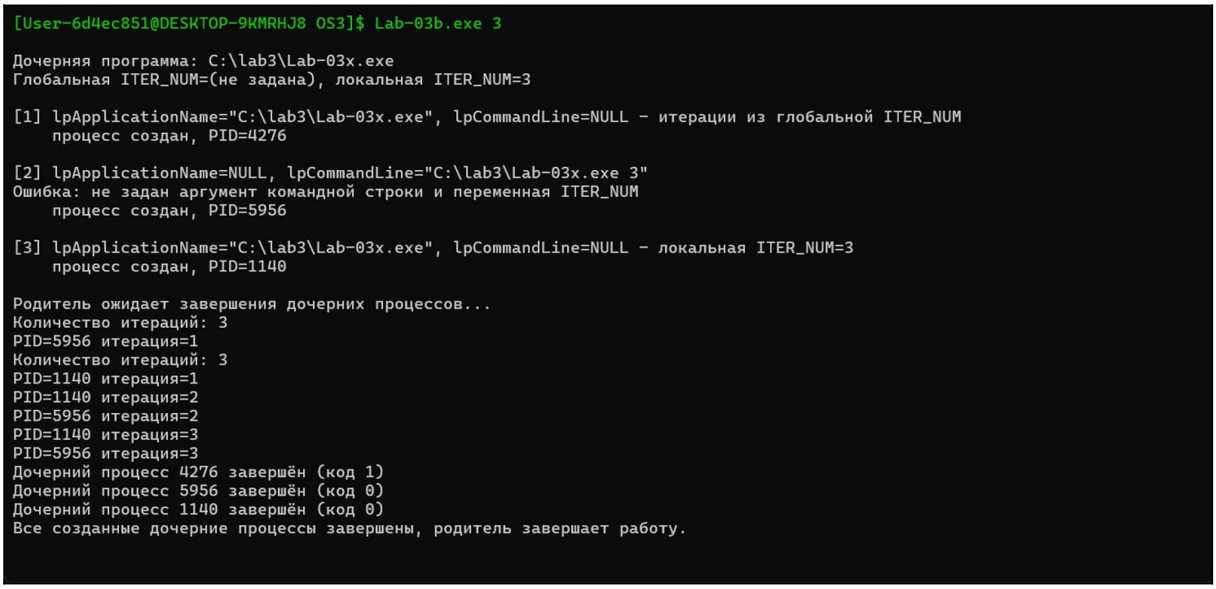


Рисунок 2.6 – *Lab*-03*b*.*exe* 3: глобальная *ITER_NUM* не задана (ошибка), аргумент работает (3), локальная *ITER_NUM*=3 работает (3)

Lab-03*c*: список процессов через *Tool Help Library* (*CreateToolhelp*32*Snapshot*, *Process*32*First*/*Next*). Вывод: имя, *PID*, *PPID*. На рисунке 2.7 – *Lab*-03*c*.*exe* (*PID*=1424) из *cmd*.*exe* (*PID*=5084). Уровень 4–5 на *Windows* выполнен.

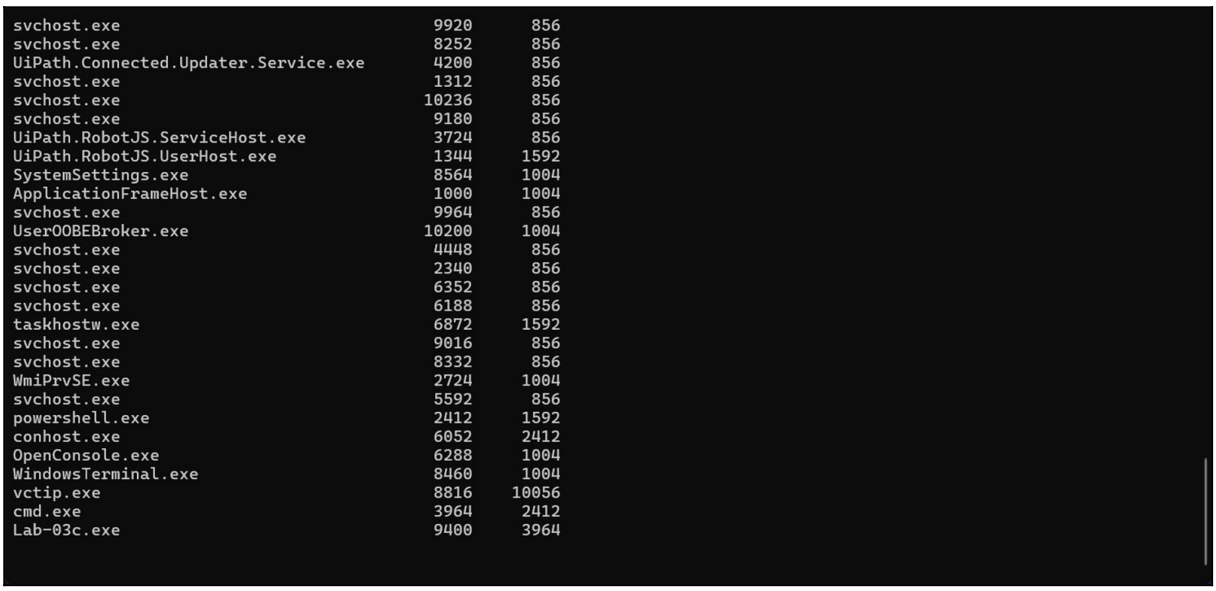


Рисунок 2.7 – *Lab*-03*c*.*exe*: таблица процессов (имя, *PID*, *PPID*), *Lab*-03*c*.*exe* (*PID*=1424), родитель *cmd*.*exe* (*PID*=5084)

Таблица процессов получена, *PID* и *PPID* верные. Скриншот трёх одновременно запущенных процессов (родитель + два дочерних) – на рисунке 2.8.

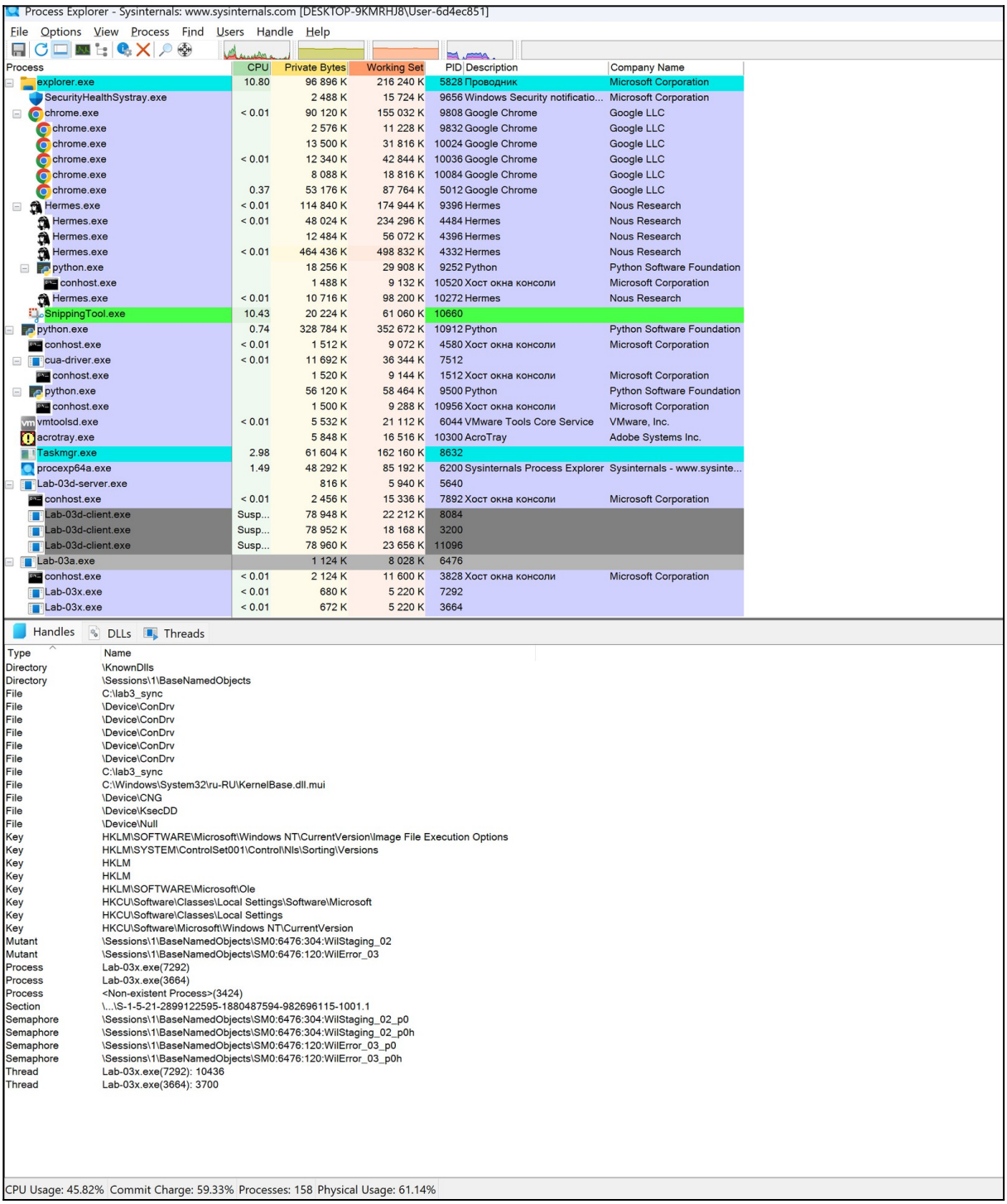


Рисунок 2.8 – Три одновременных процесса: родитель *Lab*-03*a* и два дочерних *Lab*-03*x*

Те же процессы видны из консоли командой *tasklist* (рисунок 2.9); в Диспетчере задач по фильтру находится родительский *Lab*-03*a* (рисунок 2.10). Дескрипторы родителя и двух дочерних процессов показаны в нижней панели *Handles Process Explorer* (рисунок 2.11).

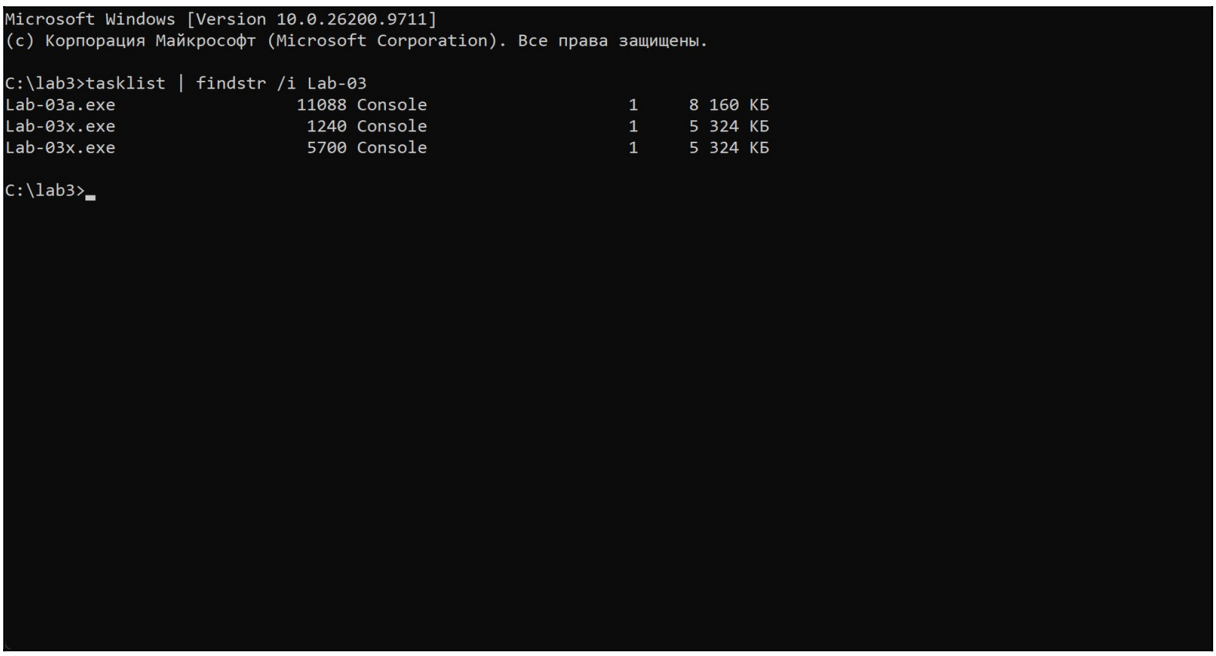


Рисунок 2.9 – *tasklist | findstr /i Lab*-03: процессы *Lab*-03*a* и два *Lab*-03*x* с *PID*

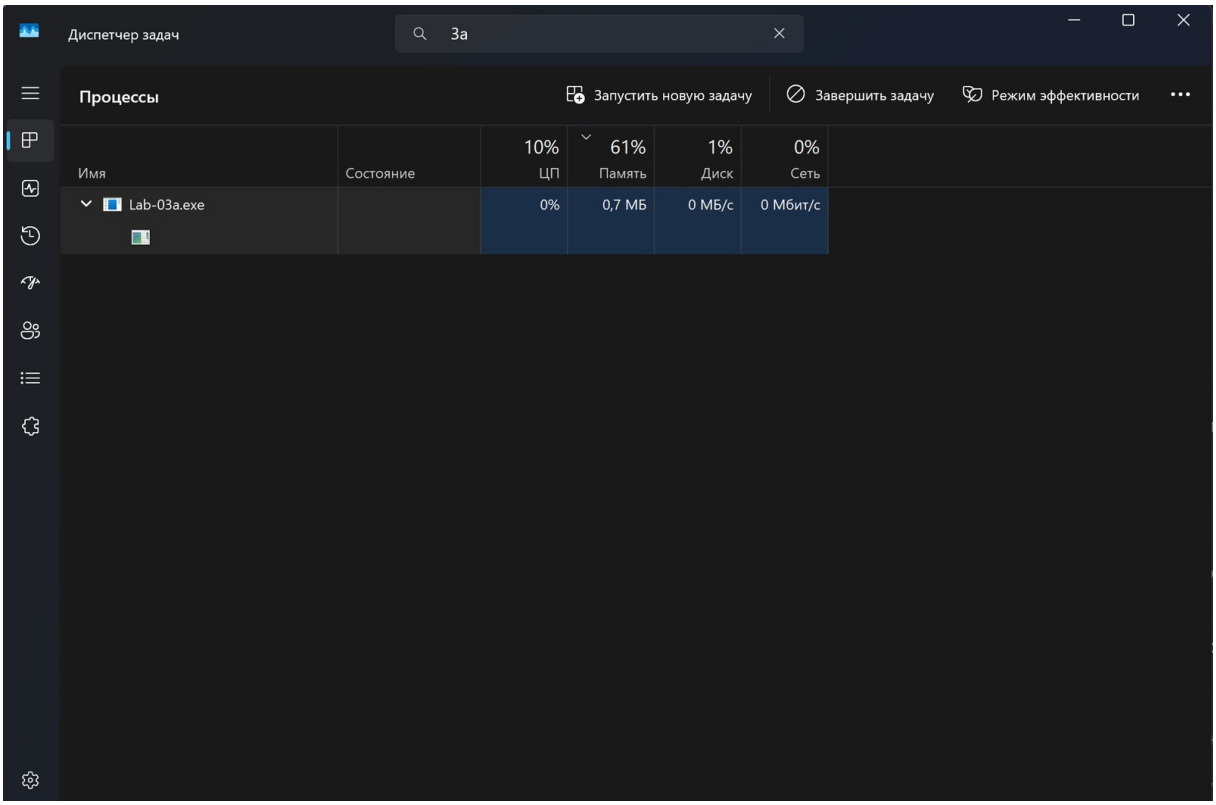


Рисунок 2.10 – Диспетчер задач *Windows*: процесс *Lab*-03*a.exe*, найденный по фильтру «3*a*»

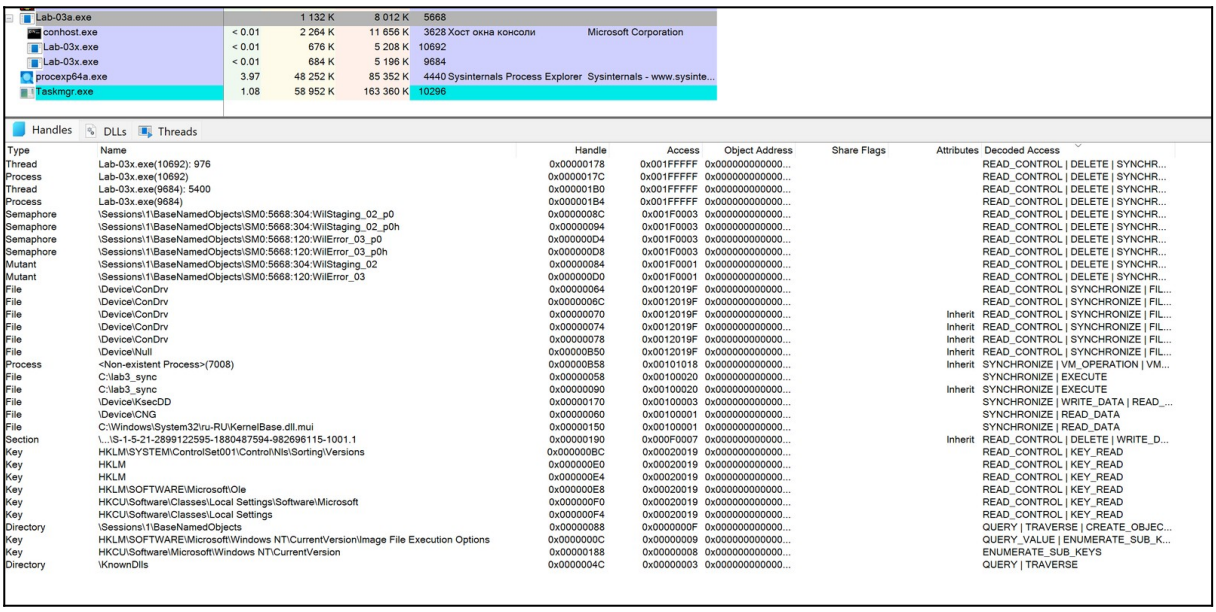


Рисунок 2.11 – *Process Explorer*: нижняя панель *Handles* процесса *Lab*-03*a* – дескрипторы дочерних *Lab*-03*x*

Уровень 4–5 на *Windows* выполнен: процессы создаются, параметры передаются, информация получена.

2.3 Уровень 7–8: *IPC* через канал и *STARTUPINFO*

Lab-03*d-client*: поиск взаимно простых с *M* в [*L*, *R*], результат в *stdout* (в *pipe*) формате *CHUNK <PID> <count>* 0 <числа>. *Lab*-03*d-server*: *CreatePipe* (*bInheritHandle=TRUE*), дескриптор записи в *STARTUPINFO* (*hStdOutput*, *STARTF_USESTDHANDLES*), делит диапазон на *K* частей, запускает *K* клиентов, собирает результаты, выводит *M*, [*L*, *R*], *K*, количество, числа, время (*QueryPerformanceCounter*). Результаты на рисунках 2.12, 2.13, 2.14.

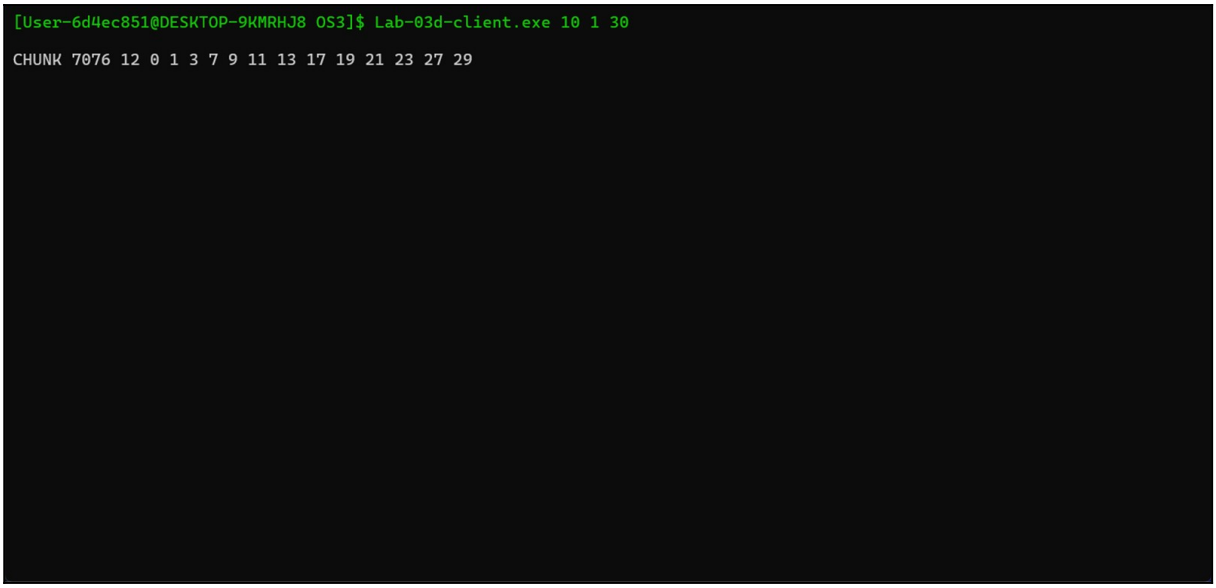


Рисунок 2.12 – *Lab*-03*d-client.exe* 10 1 30: найдены числа, вывод в канал

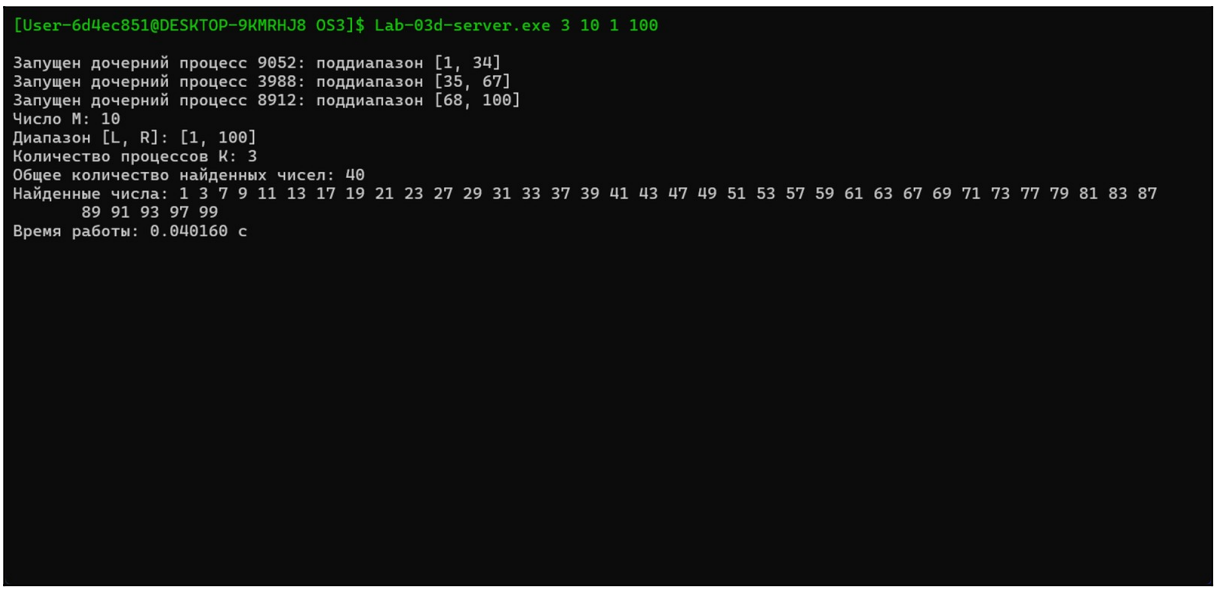


Рисунок 2.13 – *Lab*-03*d-server.exe* 3 10 1 100: *K*=3, 40 чисел, время 0.035 с

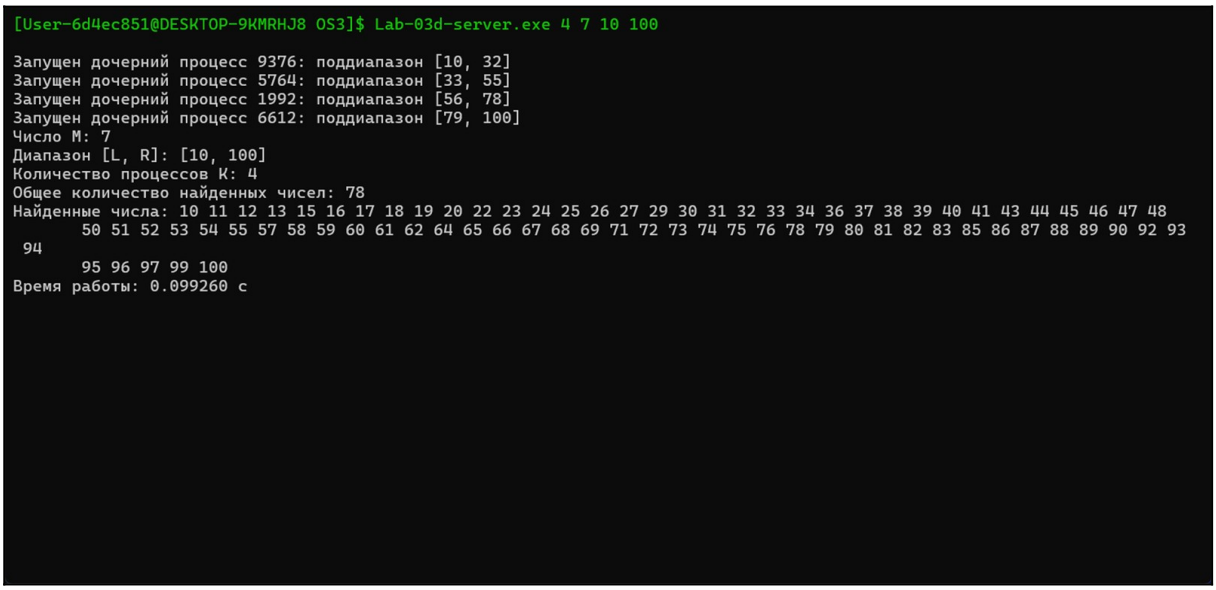


Рисунок 2.14 – *Lab*-03*d-server.exe* 4 7 10 100: *K*=4, другой *M* и диапазон

В *Process Explorer* у детей виден дескриптор *File* (*Pipe*) – анонимный канал. Сервер ждёт всех детей (*WaitForMultipleObjects*).

Нижняя панель *Handles* в *Process Explorer* показывает и дескриптор канала типа *File* (*\Device\NamedPipe*), и дескрипторы запущенных процессов *Lab*-03*d-client* (рисунок 2.15).

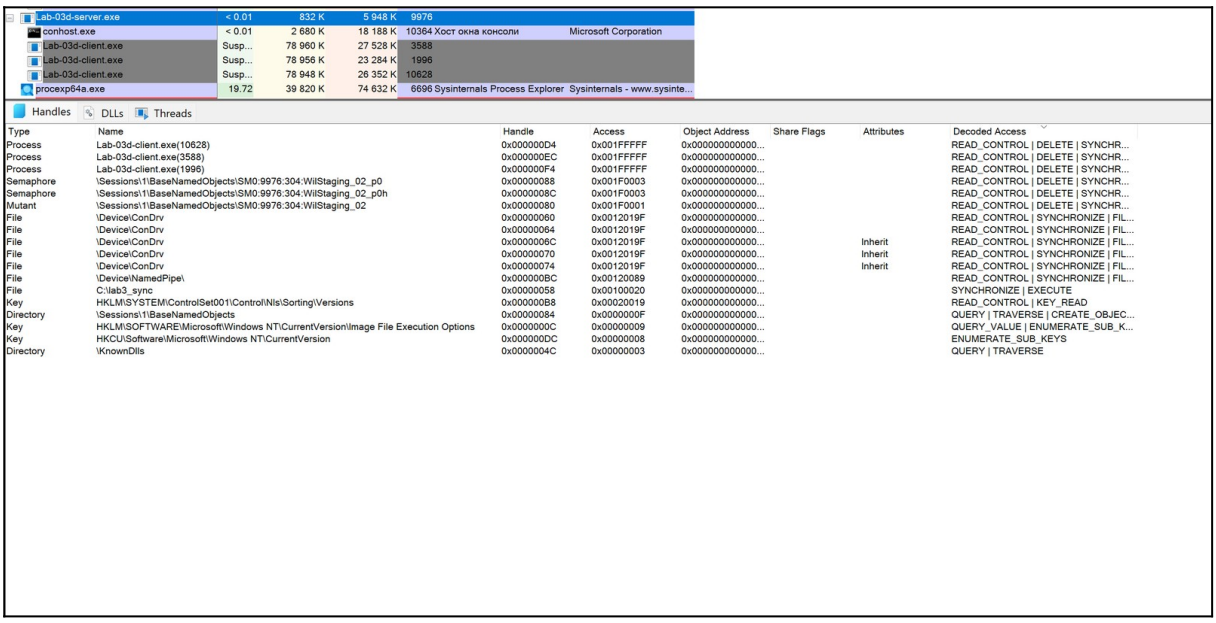


Рисунок 2.15 – *Process Explorer*: дескриптор канала *\Device\NamedPipe* и дескрипторы процессов *Lab-*03*d-client* в нижней панели *Handles*

Уровень 7–8 на *Windows* выполнен: канал анонимный, дескриптор наследуется потомком, сервер дождался всех детей и собрал результат.

2.4 Уровень 9–10: передача через *STARTUPINFO*

Задача: родитель передаёт дочернему «*Hello, my dear child*!» через *STARTUPINFO*, без окружения, аргументов, перенаправления, *IPC*. Решение: *si.lpTitle* = фраза; *CreateProcess* с *CREATE_NEW_CONSOLE* – только при новой консоли *Windows* применяет *lpTitle* как заголовок окна. Дочерний читает через *GetConsoleTitle* и выводит. Демонстрация на рисунке 2.16. Список *exe* на рисунке 2.17.



Рисунок 2.16 – *Lab-*03*hello-parent.exe*: родитель создаёт дочерний с *CREATE_NEW_CONSOLE* и *lpTitle*; дочернее окно показывает фразу

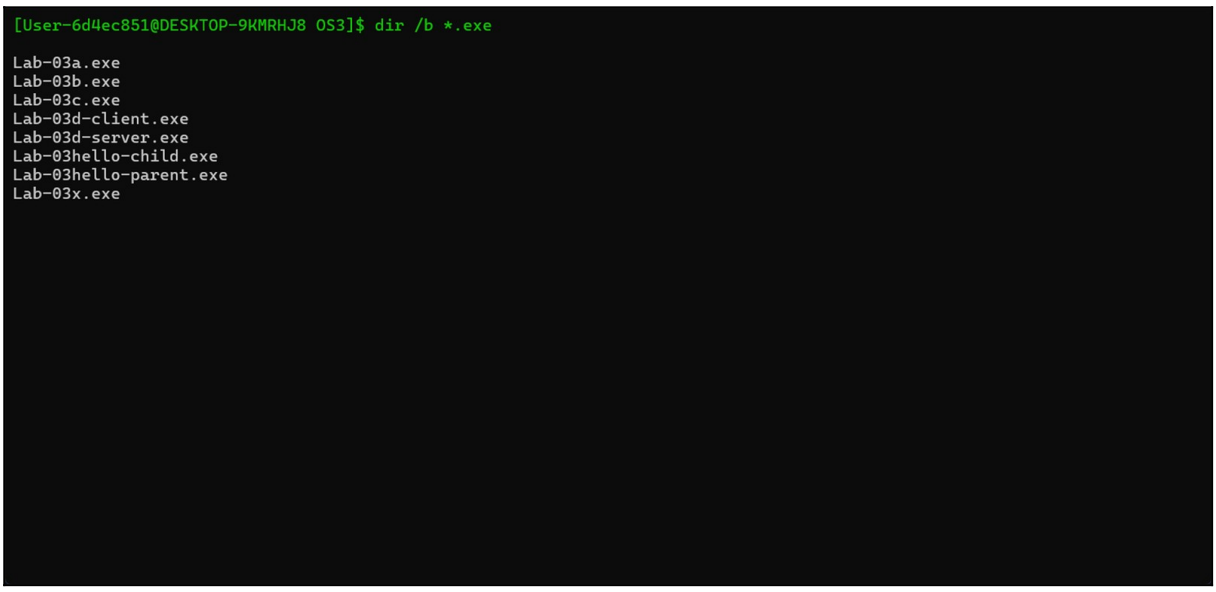


Рисунок 2.17 – Список собранных *exe* в *C*:*lab*3

Подтверждено: *lpTitle* работает только с *CREATE_NEW_CONSOLE*; аргументы в *lpApplicationName* не работают (ошибка 2); только *lpCommandLine* или *lpApplicationName*+*lpCommandLine* (с пробелом) корректны; наследуемые дескрипторы *pipe* через *STARTUPINFO* (*STARTF_USESTDHANDLES*). Уровень 9–10 на *Windows* выполнен.

2.5 Вывод по разделу

На *Windows* реализованы: *Lab*-03*x* (*ExitProcess*, аргументы/*ITER_NUM*), *Lab*-03*a* (три *CreateProcess* разными способами), *Lab*-03*b* (глобальная/локальная *ITER_NUM*), *Lab*-03*c* (*Tool Help Library*, *PID*/*PPID*), *Lab*-03*d-client*/*server* (*CreatePipe* + *STARTUPINFO.hStdOutput*, наследуемые дескрипторы), *Lab*-03*hello-parent*/*child* (*lpTitle* + *CREATE_NEW_CONSOLE*). Все собраны *MSVC ARM*64, скриншоты сделаны. Все уровни (4–5, 7–8, 9–10) выполнены.

Заключение

Лабораторная работа №3 выполнена полностью на уровне 9–10 для обеих систем.

Linux: *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*d-client*/*server*, *Lab*-03*sig*. Все собраны *GCC* 16.1.1, протестированы.

Windows: *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*b*, *Lab*-03*c*, *Lab*-03*d-client*/*server*, *Lab*-03*hello-parent*/*child*. Все собраны *MSVC ARM*64, протестированы.

Выводы по *Windows API*: *lpApplicationName* не работает (ошибка 2); *lpCommandLine* или *lpApplicationName*+*lpCommandLine* (с пробелом) – ок; *lpTitle* только с *CREATE_NEW_CONSOLE*; наследуемые дескрипторы *pipe* через *STARTUPINFO* (*STARTF_USESTDHANDLES*).

Все скриншоты и выводы сохранены в репозитории: *lab*3/*linux*/скрины/уровень_*/, *lab*3/*linux*/выводы/уровень_*/, *lab*3/*windows*/скрины/уровень_*/, *lab*3/*windows*/выводы/уровень_*/. Исходники в *lab*3/*linux*/ и *lab*3/*windows*/.